

SUPER KB-H

VERSION: 8.4

STATUS: ACTIVE

SCOPE: CANONICAL MEMORY, MUTATION PERSISTENCE, DURABILITY, STORAGE OF STRUCTURAL + RELATIONSHIP OUTCOMES ONLY

---

PURPOSE (LOCKED)

H governs canonical persistence.

H determines:

- what is durably written to canonical memory
- how structural and relationship outcomes are persisted
- how mutations are recorded and tracked over time
- how canonical state remains non-destructive and recoverable

H ensures:

- persistence is durable
- persistence is non-destructive
- persistence is idempotent
- persistence is attributable to governed execution

H does not:

- classify inputs (MC)

- control execution or retry (MS)
  - determine workflow or stage progression (D)
  - construct structural truth or proof (Z)
  - evaluate significance (MRS)
  - define reporting, trust stats, delta stats, or disclosure (DB)
  - render output (UX)
  - generate actions or behavioral outcomes (A / B)
- 

## AUTHORITY MODEL (LOCKED)

H is the sole owner of:

- canonical memory updates
- mutation persistence
- mutation accounting and tracking
- storage of structural outcomes
- storage of relationship outcomes

No other module may:

- define persistence invariants
- silently overwrite canonical state
- delete canonical artifacts
- persist state without governed attribution

Violation constitutes a deterministic system defect.

---

## CANONICAL PERSISTENCE MODEL (LOCKED)

Canonical memory is ledger-backed.

If it is not written to canonical memory, it does not exist.

All canonical writes MUST:

- be append-only or versioned
- preserve prior state
- include provenance (source, execution context, timestamp)
- be attributable to a governed execution

Canonical memory MUST:

- survive time and inactivity
  - remain recoverable
  - remain consistent under reprocessing
- 

## NON-DESTRUCTIVE INVARIANT (LOCKED)

Canonical memory MUST NOT:

- delete prior records

- silently overwrite prior records

All changes MUST be represented as:

- new entries

OR

- explicit state transitions

Loss of historical state constitutes a system defect.

---

## MUTATION PERSISTENCE (LOCKED)

H exclusively governs mutation persistence.

Mutation includes:

- baseline refresh changes
- structural updates
- relationship updates
- attribute updates

H MUST:

- persist mutation events
- maintain linkage between prior state and new state
- ensure mutations are attributable to a specific governed execution

H MUST NOT:

- interpret mutation significance
  - report mutation meaning
  - compute delta statistics for presentation
- 

## MUTATION ACCOUNTING (LOCKED)

H MUST ensure mutations are:

- traceable to execution
- attributable to canonical artifacts
- reconstructable over time

H MUST support:

- derivation of mutation history from canonical artifacts
- reconstruction of prior states

H does not:

- define trust metrics (DB owns)
  - define delta presentation (DB owns)
- 

## STORAGE OF STRUCTURAL OUTCOMES (LOCKED)

H persists outputs produced by Z.

H MUST:

- store structural artifacts written by Z
- preserve structural truth as written
- maintain provenance of structural ingestion

H MUST NOT:

- reinterpret structural truth
- validate structural correctness
- construct structural artifacts

Z writes.

H persists.

PERSISTENCE ADMISSIBILITY GATE (LOCKED)

H MUST persist only canonical outputs that have already been lawfully authorized upstream for canonical write.

H MUST NOT persist as canonical memory any output that is:

- provisional
- partial
- retry-eligible
- continuation-pending
- proof-incomplete
- reconciliation-incomplete

- non-authoritative for the active governed execution

For structural outcomes, H MUST treat Z-emitted canonical structural write payloads as the only admissible structural persistence basis.

#### Z-PROOF PERSISTENCE CHAIN LOCK (LOCKED)

For structural baseline seed, baseline enhancement, structural reconciliation, or structural mutation persistence, H MUST persist only values that are present in the Z-emitted canonical structural write payload for the same governed execution.

H MUST NOT:

- persist structural proof values not supplied by Z
- persist record counts inferred from DB reports, MS status, trace output, memory recall, prior reports, or user statements
- treat `baseline_active` as TRUE unless Z canonical structural write payload and H persistence requirements are both satisfied
- alter, normalize, or substitute Z proof/count values during persistence
- collapse current execution proof into prior cumulative baseline state without execution attribution

For any persisted structural outcome, H MUST preserve enough provenance to reconstruct:

- Z proof origin
- active artifact path
- governed execution id
- structural phase class

- persisted record count basis
- canonical write family

If required Z proof origin or execution attribution is absent, H MUST prohibit canonical persistence.

If H persists baseline, count, reconciliation, trust, delta, or mutation values not value-identical to the Z canonical structural write payload for the same governed execution:

terminal\_state: MODULE\_EXECUTION\_FAILED

failure\_class: H\_Z\_PROOF\_CHAIN\_BREACH

END Z-PROOF PERSISTENCE CHAIN LOCK

If a structural or relationship output has not yet reached its upstream authoritative closure posture for the active governed execution, H MUST:

- prohibit canonical persistence
- preserve no substitute canonical write
- treat the attempted persistence as non-compliant

Failure constitutes:

failure\_class: EXECUTION\_CONTRACT\_VIOLATION

END PERSISTENCE ADMISSIBILITY GATE (LOCKED)

OPERATOR ANCHOR PERSISTENCE PROTECTION (LOCKED)



The system operates relative to a single Operator Anchor identity.

H MUST enforce that:

- the Operator Anchor identity is not persisted as a newly discovered canonical entity
- any structural or relationship reference resolving to the Operator Anchor links to the existing canonical identity
- no duplicate canonical entity is created for the Operator Anchor

If a persistence attempt would create a canonical entity that resolves to the Operator Anchor:

H MUST:

- suppress entity creation
- allow reference linkage only

Violation constitutes:

terminal\_state: MODULE\_EXECUTION\_FAILED

reason: OPERATOR\_ENTITY\_DUPLICATION

---

STORAGE OF RELATIONSHIP OUTCOMES (LOCKED)

H persists outputs produced by MRS and related routed execution.

H MUST:

- persist RI outcomes (RI\_POSITIVE / RI\_NEGATIVE)
- persist relationship-bearing artifacts
- persist interaction evidence when routed

H MUST NOT:

- evaluate significance
- redefine RI outcomes
- interpret relationship meaning

MRS determines.

H persists.

---

## IDEMPOTENCY & RECOVERY (LOCKED)

Reprocessing identical inputs MUST NOT:

- create duplicate canonical entities
- remap stable identifiers
- alter prior canonical state unintentionally

If conflicts occur:

- resolution MUST be deterministic
- resolution MUST be recorded as a canonical event

Silent correction is prohibited.

## STABLE IDENTIFIER INVARIANT (LOCKED)

Canonical entities and events **MUST** maintain stable identifiers across executions.

Identifiers **MUST**:

- remain consistent across reprocessing of identical inputs
- not be remapped or reassigned without explicit canonical resolution
- support deterministic identity resolution

If identity conflicts occur:

- resolution **MUST** be deterministic
- resolution **MUST** be recorded as a canonical event
- silent identifier reassignment is prohibited

Creation of duplicate canonical entities for the same identity constitutes:

failure\_class: EXECUTION\_CONTRACT\_VIOLATION

---

## EXECUTION ATTRIBUTION (LOCKED)

All canonical writes **MUST** be attributable to:

- the active governed execution
- a specific execution context

Canonical artifacts MUST support:

- execution linkage
- provenance tracking
- reconstruction of what was written during a given execution

H enforces attribution.

H does not expose reporting.

#### CANONICAL WRITE FAMILY NORMALIZATION (LOCKED)

For every governed canonical write, H MUST preserve the write family of the persisted artifact or event.

At minimum, admissible write families are:

- STRUCTURAL\_BASELINE\_SEED
- STRUCTURAL\_BASELINE\_ENHANCEMENT
- STRUCTURAL\_RECONCILIATION\_ARTIFACT
- STRUCTURAL\_MUTATION\_EVENT
- RELATIONSHIP\_OUTCOME
- INTERACTION\_EVIDENCE\_LINKAGE

H MUST ensure that canonical persistence for the active governed execution supports derivation of:

- what write family occurred
- whether the write established baseline state, enhanced baseline state, or only recorded reconciliation / mutation / relationship outcome

- which persisted artifacts belong to the same governed execution

H MUST NOT collapse distinct write families into an undifferentiated cumulative persistence record.

If canonical persistence occurs without a determinable write family for the active governed execution:

failure\_class: EXECUTION\_CONTRACT\_VIOLATION

END CANONICAL WRITE FAMILY NORMALIZATION (LOCKED)

---

EXECUTION-SCOPED PERSISTENCE REQUIREMENT (LOCKED)

When mutation or persistence occurs during a governed execution:

Canonical artifacts MUST support derivation of:

- what was written in this execution
- what changed in this execution

This requirement applies to:

- structural ingestion
- baseline refresh
- RI execution
- relationship updates

Global cumulative state alone is insufficient.

BASELINE RECONCILIATION PERSISTENCE (LOCKED)

PHASE-AWARE BASELINE PERSISTENCE (LOCKED)

H MUST preserve the distinction between:

- initial baseline creation
- baseline enhancement / replacement
- enhancement-only supplementation that is not baseline-authoritative identity replacement

Accordingly:

- an initial authoritative baseline seed write MUST be persistable as `STRUCTURAL_BASELINE_SEED`
- a baseline-present LinkedIn enhancement / replacement write MUST be persistable as `STRUCTURAL_BASELINE_ENHANCEMENT` together with reconciliation artifacts and mutation events
- enhancement-only non-authoritative supplementation MUST NOT be persisted as fresh baseline creation or as authoritative identity-field replacement

H MUST ensure that later reconstruction can determine whether a governed execution:

- established the canonical baseline for the first time
- enhanced an already established canonical baseline
- supplemented canonical memory without replacing authoritative baseline identity state

If H cannot preserve this distinction for the active governed execution, canonical persistence is non-compliant.

Failure constitutes:

failure\_class: `EXECUTION_CONTRACT_VIOLATION`

END PHASE-AWARE BASELINE PERSISTENCE (LOCKED)

When a governed execution performs a baseline refresh or replacement:

H MUST ensure that canonical persistence includes reconciliation artifacts representing the change between prior and current state.

Requirements:

- The prior baseline state MUST remain recoverable
- The new baseline state MUST be written without overwriting prior state
- All changes between baseline states MUST be represented as canonical mutation events
- Reconciliation artifacts MUST be attributable to the active governed execution

Reconciliation artifacts MUST support derivation of:

- prior baseline state
- current baseline state
- mutation events linking prior and current state

Prohibited behavior:

- silent overwrite or replacement of prior baseline state
- reliance on in-memory or inferred comparisons without canonical persistence
- mutation representation without execution attribution

If a baseline refresh occurs without canonical reconciliation artifacts:

H MUST treat the operation as non-compliant.

Failure constitutes:

failure\_class: EXECUTION\_CONTRACT\_VIOLATION

AUTHORITATIVE-SOURCE CONTAINMENT (LOCKED)

H MUST NOT promote non-authoritative, pass-through-only, preview-derived, or enrichment-only artifacts into canonical baseline-authority state.

Accordingly, H MUST NOT:

- persist preview-like or partial artifact outputs as authoritative canonical memory
- persist pass-through-only artifacts as baseline-authoritative identity writes
- persist enrichment-only non-LinkedIn artifact paths as authoritative LinkedIn identity replacement
- convert downstream-readable supplementation into authoritative baseline creation or authoritative baseline overwrite

If an upstream-governed artifact path is preserved as pass-through-only, enhancement-only, or non-authoritative for identity replacement, H MUST preserve that containment in canonical persistence.

Violation constitutes:

failure\_class: EXECUTION\_CONTRACT\_VIOLATION

END AUTHORITATIVE-SOURCE CONTAINMENT (LOCKED)

---

NON-OWNERSHIP BOUNDARY (LOCKED)

H MUST NOT:

- define reporting structures (DB owns)



- define trust or delta statistics (DB owns)
- control execution or retry (MS owns)
- define workflow progression (D owns)
- render output (UX owns)
- evaluate significance (MRS owns)

H determines only:

- what is persisted
- how it is persisted
- how it remains durable and recoverable

---

HANDOFF CONTRACT (LOCKED)

H receives:

- structural outputs from Z
- relationship outcomes from MRS
- interaction evidence from I when routed
- routed execution results

H provides:

- canonical persistence
- mutation tracking
- execution-attributed storage

H MUST NOT:

- modify upstream outputs
- inject interpretation
- define downstream reporting

---

## FAILURE CONDITIONS (LOCKED)

H is non-compliant if:

- canonical state is overwritten or deleted
- persistence occurs without provenance
- persistence is not attributable to execution
- duplicate entities are created from identical inputs
- mutation history cannot be reconstructed
- canonical state diverges under identical input processing

Violation results in:

terminal\_state: MODULE\_EXECUTION\_FAILED

---

## DESIGN CONSTRAINT (LOCKED)

H is a persistence engine only.

H exists to answer:

- What was written?
- What changed?
- Can it be proven and reconstructed?

H must never:

- interpret
- report
- decide
- render

Any expansion beyond persistence constitutes a system defect.